

## Module - II

### Regular Expressions and Regular Languages

1

#### 2.1 Introduction to Regular Expressions

- A Regular Expression (RE) is an algebraic notation used to denote a **regular language**.
- It provides a compact and mathematical way to describe a **set of strings** over an alphabet.
- Every regular expression denotes a **regular language**, and every regular language can be denoted by some regular expression.
- Regular expressions and finite automata are **equivalent** in expressive power.

##### Key Idea :

Regular expressions describe patterns in strings using a small set of operators. They are widely used in compiler design, text processing, search tools and specification of simple languages.

#### 2.2 Alphabet, String and Language (Recap)

- Alphabet ( $\Sigma$ )**: A finite non-empty set of symbols.  
Example:  $\Sigma = \{0,1\}$ ,  $\Sigma = \{a,b,c\}$
- String (word)**: A finite sequence of symbols from  $\Sigma$ .  
Example:  $w = 0101$ ,  $w = aab$
- Length of a string**: Number of symbols in the string, Denoted by  $|w|$ . Example:  $|1010| = 4$
- Empty string ( $\epsilon$ )**: A string of length zero.  $\epsilon \in \Sigma^*$ .
- Empty language ( $\emptyset$ )**: A language with no strings.  $\emptyset \subseteq \Sigma^*$ .
- Language (L)**: A set of strings over  $\Sigma$ .  $L \subseteq \Sigma^*$ .

##### Examples :

- $\Sigma = \{0,1\}$
- $L = \{w \in \{0,1\}^* \mid w \text{ ends with } 0\}$
- Some strings in L: 0, 10, 110, 1010
- Strings not in L:  $\epsilon$ , 1, 11, 101

#### 2.3 Formal Definition of Regular Expression

A regular expression over an alphabet  $\Sigma$  is defined recursively as follows :

- $\emptyset$  is a regular expression and denotes the empty language  $L(\emptyset) = \emptyset$ .
- $\epsilon$  is a regular expression and denotes  $\{\epsilon\}$ .
- For each  $a \in \Sigma$ ,  $a$  is a regular expression and denotes the language  $\{a\}$ .
- If  $r$  and  $s$  are regular expressions denoting  $L(r)$  and  $L(s)$ , then  $(r + s)$ ,  $(rs)$  and  $(r^*)$  are also regular expressions denoting  $L(r) \cup L(s)$ ,  $L(r)L(s)$ , and  $(L(r))^*$  respectively.

#### 2.4 Basic Operators in Regular Expressions

| Operator      | Symbol        | Description         | Meaning (Language) | Example                            |
|---------------|---------------|---------------------|--------------------|------------------------------------|
| Union         | $+$ (or $ $ ) | either $r$ or $s$   | $L(r) \cup L(s)$   | $a + b = \{a, b\}$                 |
| Concatenation | $rs$          | $r$ followed by $s$ | $L(r)L(s)$         | $ab = \{ab\}$                      |
| Kleene Star   | $r^*$         | zero or more        | $(L(r))^*$         | $a^* = \{\epsilon, a, aa, \dots\}$ |

#### 2.5 Precedence and Parentheses

- Kleene star ( $*$ ) has the **highest precedence**.
- Concatenation** has higher precedence than union ( $+$ ).
- Use parentheses to avoid ambiguity.

##### Precedence order :

- $*$  (highest)
- Concatenation
- $+$  (lowest)

##### Examples :

- $a + bc = a + (bc)$  (not  $(a + b)c$ )
- $a^*b = (a^*)b$
- $(a + b)^*c = ((a + b)^*)c$

##### Common Mistake :

- $a + b^*$  means  $a + (b^*)$ ,
- not  $(a + b)^*$ .

#### 2.6 Languages Denoted by Basic Regular Expressions

| Regular Expression | Language $L(r)$                                 | Example Strings                 | Not in $L(r)$         |
|--------------------|-------------------------------------------------|---------------------------------|-----------------------|
| $\emptyset$        | $\emptyset$                                     | (none)                          | $\epsilon, a, 0$      |
| $\epsilon$         | $\{\epsilon\}$                                  | $\epsilon$                      | $a, 0, 1$             |
| $a$                | $\{a\}$                                         | $a$                             | $\epsilon, b, 0$      |
| $a + b$            | $\{a, b\}$                                      | $a, b$                          | $\epsilon, ab$        |
| $ab$               | $\{ab\}$                                        | $ab$                            | $a, b, \epsilon$      |
| $a^*$              | $\{\epsilon, a, aa, \dots\}$                    | $\epsilon, a, aa, aaa$          | $b, ab$               |
| $(a + b)^*$        | all strings over $\{a, b\}$                     | $\epsilon, a, b, ab, ba, \dots$ | (none)                |
| $1(0+1)^*0$        | binary strings starting and ending with 1 and 0 | $10, 110, 1010$                 | $\epsilon, 1, 0, 111$ |

#### 2.7 Examples of Regular Expressions and their Languages

| RE          | Language Description                          | Language L                                                                   | Sample Strings                  | Not in L             |
|-------------|-----------------------------------------------|------------------------------------------------------------------------------|---------------------------------|----------------------|
| $0 + 1$     | either 0 or 1                                 | $\{0,1\}$                                                                    | 0, 1                            | $\epsilon, 00$       |
| $(0+1)^*$   | all binary strings                            | $\{0,1\}^*$                                                                  | $\epsilon, 0, 1, 01, 1101$      | (none)               |
| $00 + 11$   | exactly 00 or 11                              | $\{00, 11\}$                                                                 | 00, 11                          | 0, 1, 01             |
| $(0+1)^*0$  | all strings ending with 0                     | $\{w \in \{0,1\}^* \mid w \text{ ends with } 0\}$                            | 0, 10, 110, 1010                | $\epsilon, 1, 11$    |
| $0^*1^*$    | zero or more 0's followed by zero or more 1's | $\{0^m1^n \mid m, n \geq 0\}$                                                | $\epsilon, 0, 00, 1, 0011, 111$ | 10, 01, 101          |
| $(ab)^*$    | any number of repetitions of ab               | $\{\epsilon, ab, abab, \dots\}$                                              | $\epsilon, ab, abab$            | a, b, aba            |
| $a(0+1)^*b$ | strings starting with a and ending with b     | $\{w \in \{0,1\}^* \mid w \text{ starts with } a \text{ and ends with } b\}$ | ab, a0b, a101b                  | $\epsilon, a, b, ba$ |



## Module - II

### Regular Expressions and Regular Languages

3

#### 2.8 More Examples and Practice

##### Example 1 :

Regular expression:  $(0 + 1)^*01$   
 Language : All binary strings ending with 01.  
 Example strings : 01, 001, 1101, 00001  
 Not in L :  $\epsilon$ , 0, 10, 101, 100

##### Example 2 :

Regular expression:  $0^*(1 + 01)^*$   
 Meaning : Any number of 0's followed by any number of 1 or 01.  
 Example strings :  $\epsilon$ , 0, 00, 1, 01, 001, 0101  
 Not in L : 10, 110 (start with 1)

##### Example 3 :

Regular expression:  $(a + b)^*abb$   
 Meaning : All strings over  $\{a, b\}$  ending with abb.  
 Example strings : abb, aabb, babb, aaabb  
 Not in L :  $\epsilon$ , ab, aab, aba

##### Example 4 :

Regular expression:  $(0 + 1)0(0 + 1)$   
 Meaning : All binary strings of length 3 with middle symbol 0.  
 Example strings : 000, 010, 100, 110  
 Not in L : 001, 101, 111 (middle not 0)

#### 2.9 Identities and Simplification Techniques

| No. | Identity                               | Explanation                                        |
|-----|----------------------------------------|----------------------------------------------------|
| 1   | $r + \emptyset = r$                    | Union with empty set does nothing.                 |
| 2   | $r\epsilon = r = \epsilon r$           | Concatenation with $\epsilon$ does nothing.        |
| 3   | $r + r = r$                            | Idempotent law.                                    |
| 4   | $(r + s) = (s + r)$                    | Commutative law (union).                           |
| 5   | $((r + s) + t) = (r + (s + t))$        | Associative law (union).                           |
| 6   | $(rs)t = r(st)$                        | Associative law (concatenation).                   |
| 7   | $r(s + t) = rs + rt$                   | Distributive law.                                  |
| 8   | $\emptyset r = r\emptyset = \emptyset$ | Concatenation with $\emptyset$ gives $\emptyset$ . |
| 9   | $(r^*)^* = r^*$                        | Kleene star idempotent.                            |
| 10  | $\epsilon + r r^* = r^*$               | Useful for simplification.                         |

##### Example of Simplification

Simplify :  $(0 + 0)(1 + \epsilon)^*$   
 $= 0(1 + \epsilon)^*$  [since  $0 + 0 = 0$ ]  
 $= 0(1 + \epsilon)^*$   
 $= 0(1^*)$  [since  $1 + \epsilon = 1^*$ ]  
 $= 01^*$

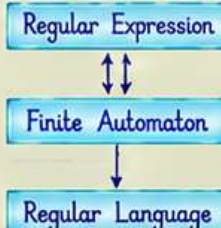
##### Checkpoint

- What language is denoted by  $(0 + 1)^*00$ ?
- Simplify :  $(a + a)(b + \epsilon)$ .
- Give a regular expression for all strings over  $\{a, b\}$  that start with a and end with b.

#### 2.10 Regular Expressions and Finite Automata

- Regular expressions and finite automata are **equivalent** in expressive power.
- For every regular expression, there exists a finite **automaton** (usually an NFA) that accepts the same **language**.
- For every finite automaton (DFA/NFA), there exists a regular expression that denotes the same language.

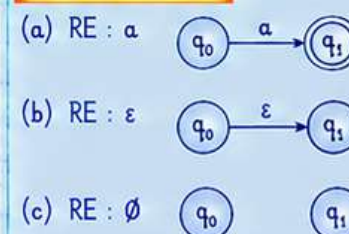
##### Equivalence :



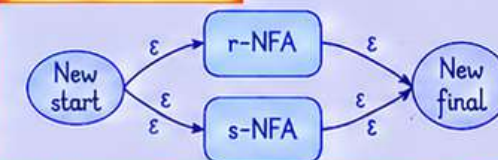
#### 2.11 Constructing an NFA from a Regular Expression (Thompson's Construction)

- We can systematically construct an NFA for any regular expression using simple rules.
- Each operator (union, concatenation, star) has a standard construction.
- The resulting NFA may have  $\epsilon$ -transitions.

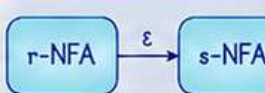
##### (1) Basic Cases



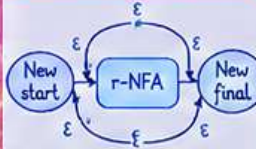
##### (2) Union : $r + s$



##### (3) Concatenation : $rs$

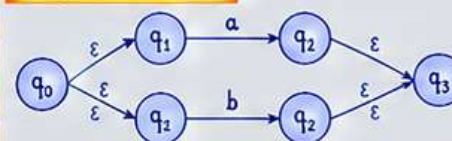


##### (4) Kleene Star : $r^*$



#### 2.12 Worked Example : Construct NFA for $(a + b)^*abb$

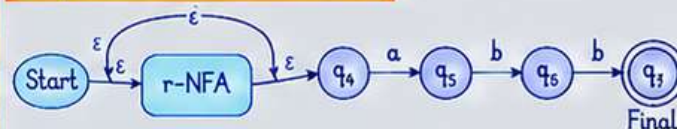
##### Step 1 : $(a + b)$



##### Step 2 : $(a + b)^*$



##### Step 3 : Concatenate with abb



##### Language :

All strings over  $\{a, b\}$  ending with abb.



## Regular Expressions and Regular Languages

## 2.13 Converting a DFA to a Regular Expression (State Elimination Method)

- Given a DFA, we can systematically convert it to a regular expression using the state elimination method.
- Idea** : Repeatedly eliminate intermediate states and update the transition labels using regular expression operations.
- At the end, we get a single expression from start state to final state.
- This expression denotes the same language as the given DFA.

## Algorithm :

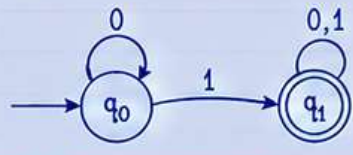
- Add new start state with  $\epsilon$ -transition to original start.
- Add new final state with  $\epsilon$ -transition from all original final states.
- For each intermediate state  $k$ , update transitions using :  

$$R_{ij} = R_{ij} + R_{ik}(R_{kk})^*R_{kj}$$
- Remove state  $k$  and repeat until only start and final remain.
- The label from start to final is the required regular expression.

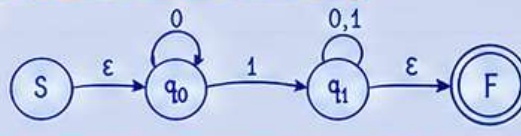
## Key Idea :

- Each state elimination introduces union, concatenation and star operations.
- We keep updating the transitions using regular expression identities.
- The order of elimination does not change the final language (may give different expressions).

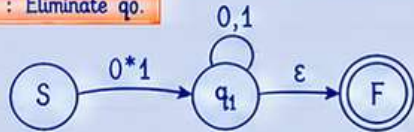
**Example :** Convert the following DFA to a regular expression.



Step 1 : Add new start and final states.



Step 2 : Eliminate  $q_0$ .



Step 3 : Eliminate  $q_1$  (final):

The regular expression from  $S$  to  $F$  is :

$$0^*1(0 + 1)^*$$

2.14 Comparison : RE  $\rightarrow$  NFA, DFA  $\rightarrow$  RE

| Aspect    | RE $\rightarrow$ NFA (Thompson's)                                            | DFA $\rightarrow$ RE (State Elimination)                     |
|-----------|------------------------------------------------------------------------------|--------------------------------------------------------------|
| Direction | From expression to automaton                                                 | From automaton to expression                                 |
| Result    | $\epsilon$ -NFA                                                              | Regular expression                                           |
| Method    | Construct small NFAs for operators and combine using $\epsilon$ -transitions | Repeatedly eliminate states and update transitions using REs |
| Size      | Linear in size of expression                                                 | Can be exponential in worst case                             |
| Use       | To build an automaton for pattern                                            | To find expression for a given DFA                           |

## 2.15 Applications of Regular Expressions

- Regular expressions are widely used in many areas of computer science:

- Compiler design** : lexical analysis (token recognition).
- Text processing** : search and pattern matching.
- Search tools** : **grep**, **regex** in editors, IDEs.
- Data validation** : check format of email, phone number, etc.
- Modeling simple languages** : e.g., identifiers, numbers, keywords.

## Examples of patterns :

- Identifiers :  $[a-zA-Z][a-zA-Z0-9]^*$
- Binary numbers :  $(0 + 1)^*$
- Strings ending with 01 :  $(0 + 1)^*01$
- Email-like :  $[a-zA-Z0-9]^+@[a-zA-Z0-9]^+$

## 2.16 Introduction to Regular Grammar

- A grammar is a **generative** way to describe a language.
- A **regular grammar** generates exactly the regular languages.
- It has very simple production rules, which correspond to finite automata.
- There are two types of regular grammars :

- Right Linear Grammar (RLG)** : Productions of the form  $A \rightarrow aB \mid a \mid \epsilon$
- Left Linear Grammar (LLG)** : Productions of the form  $A \rightarrow Ba \mid a \mid \epsilon$

## 2.17 Formal Definition of Regular Grammar

A regular grammar is a 4-tuple  $G = (V, \Sigma, P, S)$ , where

- $V$  : finite set of non-terminals.
- $\Sigma$  : finite set of terminals ( $\Sigma \cap V = \emptyset$ ).
- $P$  : finite set of productions.
- $S$  : start symbol ( $S \in V$ ).

## Right Linear Grammar (RLG)

Productions are of the form :  
 $A \rightarrow aB \mid a \mid \epsilon$   
 $A, B \in V, a \in \Sigma$

## Left Linear Grammar (LLG)

Productions are of the form :  
 $A \rightarrow Ba \mid a \mid \epsilon$   
 $A, B \in V, a \in \Sigma$

## 2.18 Examples of Regular Grammar

**Example 1 :** RLG for strings ending with 01

$G = (\{S, A\}, \{0, 1\}, P, S)$   
 $P :$   
 $S \rightarrow 0S \mid 1S \mid 0A$   
 $A \rightarrow 1$

**Example 2 :** LLG for strings starting with 1

$G = (\{S, A\}, \{0, 1\}, P, S)$   
 $P :$   
 $S \rightarrow A1 \mid 1$   
 $A \rightarrow A0 \mid A1 \mid \epsilon$

## 2.19 Regular Grammar and Regular Expressions

- Every regular grammar generates a regular language.
- For every regular expression, there exists a regular grammar that generates the same language.
- Regular grammars, regular expressions and finite automata are all equivalent in expressive power.

## Equivalence :

Regular Expression  
 $\updownarrow$   
 Regular Grammar  
 $\updownarrow$   
 Finite Automaton  
 $\updownarrow$   
 Regular Language

## 2.20 Converting Regular Grammar to Finite Automata

- Given a regular grammar  $G = (V, \Sigma, P, S)$ , we can construct an equivalent NFA.
- Each non-terminal becomes a state.
- For each production, we add transitions according to the form:

- $A \rightarrow aB$  : add transition  $A \xrightarrow{a} B$
- $A \rightarrow a$  : add transition  $A \xrightarrow{a} q_f$  (new final state)
- $A \rightarrow \epsilon$  : make  $A$  a final state



## Pumping Lemma and Closure Properties

### 2.21 Pumping Lemma for Regular Languages

- The pumping lemma gives a **necessary condition** for a language to be regular.
- It is mainly used to **prove** that a language is **not regular**.
- It does not provide a direct method to show that a language is regular.
- If a language is regular, then all sufficiently long strings in the language can be "pumped" (repeated) in a certain part and the resulting string also remains in the language.

#### Formal Statement :

If  $L$  is a regular language, then there exists a pumping length  $p \geq 1$  such that every string  $w \in L$  with  $|w| \geq p$  can be written as

$$w = xyz$$

such that :

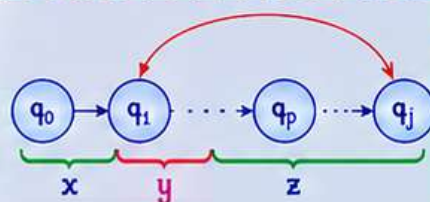
- $|y| \geq 1$  ( $y$  is non-empty)
- $|xy| \leq p$  ( $y$  appears within first  $p$  symbols)
- For all  $i \geq 0$ ,  $xy^iz \in L$  (pumping property)

#### Key Points :

- $p$  depends only on the language  $L$ , not on the particular string.
- $y$  must be non-empty.
- $y$  must lie within the first  $p$  symbols of the string.
- The string can be pumped any number of times ( $i = 0, 1, 2, \dots$ ).

### 2.22 Intuition Behind the Pumping Lemma

- A finite automaton has only a finite number of states (say  $P$  states).
- When it reads a long string ( $|w| \geq p$ ), it must visit some state more than once.
- The part of the string between the two visits to the same state can be "pumped" (repeated)
- and the automaton can still accept the string.



### 2.23 Example 1 : Prove $L = \{0^n 1^n \mid n \geq 0\}$ is not regular

#### Step 1 : Assume $L$ is regular

Then pumping length  $p$  exists.

#### Step 2 : Choose a string

$w = 0^p 1^p \in L$  and  $|w| = 2p \geq p$ .

#### Step 3 : Consider all possible decompositions

$w = xyz$ , where  $|y| \geq 1$  and  $|xy| \leq p$ .  
Since  $|xy| \leq p$ , the substring  $y$  consists only of 0's.

#### Step 4 : Pump with $i = 0$

Let  $y = 0^k$ ,  $k \geq 1$ .

Then  $xy^0z = 0^{p-k} 1^p$

which has different number of 0's and 1's, so  $xy^0z \notin L$ .

#### Step 5 : Contradiction

This contradicts the pumping lemma.

Hence  $L = \{0^n 1^n \mid n \geq 0\}$  is not regular.

### 2.24 Example 2 : Prove $L = \{0^n 1^n 0^n \mid n \geq 0\}$ is not regular

#### Step 1 : Assume $L$ is regular

Let  $p$  be the pumping length.

#### Step 2 : Choose a string

$w = 0^p 1^p 0^p \in L$

#### Step 3 : Consider all cases for $y$

Since  $|xy| \leq p$ , the substring  $y$  lies within the first  $p$  symbols, so  $y$  consists only of 0's.  
Let  $y = 0^k$ ,  $k \geq 1$ .

#### Step 4 : Pump with $i = 0$

$xy^0z = 0^{p-k} 1^p 0^p$

This string has different number of 0's in the first and last blocks, so  $xy^0z \notin L$ .

#### Step 5 : Contradiction

Hence  $L = \{0^n 1^n 0^n \mid n \geq 0\}$  is not regular.

### 2.25 Closure Properties of Regular Languages

- The class of regular languages is closed under several operations.
- If  $L_1$  and  $L_2$  are regular languages, then the resulting language obtained by applying the operation is also regular.
- These properties are very useful in proving languages to be regular.

| Operation            | Notation            | Resulting Language                         | Closed ? |
|----------------------|---------------------|--------------------------------------------|----------|
| Union                | $L_1 \cup L_2$      | $L_1 \cup L_2$                             | Yes      |
| Concatenation        | $L_1 L_2$           | $\{xy \mid x \in L_1, y \in L_2\}$         | Yes      |
| Kleene Star          | $L_1^*$             | $(L_1)^*$                                  | Yes      |
| Kleene Plus          | $L_1^+$             | $(L_1)^+$                                  | Yes      |
| Complement           | $\overline{L_1}$    | $\Sigma^* - L_1$                           | Yes      |
| Intersection         | $L_1 \cap L_2$      | $L_1 \cap L_2$                             | Yes      |
| Difference           | $L_1 - L_2$         | $L_1 - L_2$                                | Yes      |
| Reversal             | $L_1^R$             | $\{w^R \mid w \in L_1\}$                   | Yes      |
| Homomorphism         | $h(L_1)$            | $\{h(w) \mid w \in L_1\}$                  | Yes      |
| Inverse Homomorphism | $h^{-1}(L_1)$       | $\{w \mid h(w) \in L_1\}$                  | Yes      |
| Left Quotient        | $L_1 / L_2$         | $\{w \mid \exists x \in L_2, wx \in L_1\}$ | Yes      |
| Right Quotient       | $L_1 \setminus L_2$ | $\{w \mid \exists x \in L_2, wx \in L_1\}$ | Yes      |

### 2.26 Examples of Closure Properties

#### Example 1 : Union

$L_1 = \{0^*\}$  and  $L_2 = \{1^*\}$  are regular.  
 $L_1 \cup L_2 = \{0^*\} \cup \{1^*\}$  is also regular.

#### Example 2 : Concatenation

$L_1 = \{0^*\}$  and  $L_2 = \{1^*\}$  are regular.  
 $L_1 L_2 = \{0^* 1^*\}$  is regular.

#### Example 3 : Complement

$L$  = all strings over  $\{0,1\}$  ending with 0.  
 $\overline{L}$  = all strings over  $\{0,1\}$  not ending with 0  
is also regular.

#### Note :

- Regular languages form a very robust class.
- They are closed under all commonly used operations.
- This makes them extremely useful in design and analysis of simple patterns and language processing tasks.

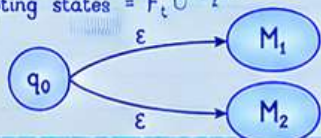


## Closure Properties of Regular Languages

### 2.27 Proof Ideas for Some Closure Properties

**(1) Union :** If  $L_1$  and  $L_2$  are regular.

- Let  $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$  for  $L_1$
- Let  $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$  for  $L_2$
- Construct DFA/NFA with new start state  $q_0$ .
- $\epsilon$ -transitions from  $q_0$  to  $q_1$  and  $q_2$ .
- Accepting states =  $F_1 \cup F_2$ .



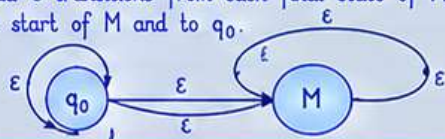
**(2) Concatenation :** If  $L_1$  and  $L_2$  are regular.

- Let  $M_1$  and  $M_2$  be NFAs for  $L_1$  and  $L_2$ .
- Add  $\epsilon$ -transitions from each final state of  $M_1$  to the start state of  $M_2$ .
- Start state = start of  $M_1$ .
- Final states = final states of  $M_2$ .



**(3) Kleene Star :** If  $L$  is regular.

- Let  $M$  be an NFA for  $L$ .
- Add new start state  $q_0$  which is also a final state.
- Add  $\epsilon$ -transitions from  $q_0$  to start of  $M$ .
- Add  $\epsilon$ -transitions from each final state of  $M$  back to start of  $M$  and to  $q_0$ .



**(4) Complement :** If  $L$  is regular.

- Let  $M$  be a complete DFA for  $L$ .
- Swap final and non-final states.
- The resulting DFA recognizes  $\Sigma^* - L$ .

**Note :** DFA must be complete (every state has transition for every symbol). If not complete, add a dead state first.

### 2.28 Non-Closure Property

Regular languages are **NOT** closed under :

- Intersection with a non-regular language.
- Difference with a non-regular language
- Shuffle
- Reversal with a non-regular language

**Example :** Intersection with a non-regular language

Let  $L_1 = \{0^*\}$  (regular)  
 $L_2 = \{0^n 1^n \mid n \geq 0\}$  (non-regular)

Then  $L_1 \cap L_2 = \{0^n 1^n \mid n \geq 0\}$  (non-regular)  
 Hence not closed under intersection with a non-regular language.

### 2.29 Important Results Summary

| Property                      | Closed under ? | Construction idea / Remark                                    |
|-------------------------------|----------------|---------------------------------------------------------------|
| Union                         | Yes            | New start state with $\epsilon$ -transitions                  |
| Concatenation                 | Yes            | $\epsilon$ -transitions from final of $M_1$ to start of $M_2$ |
| Kleene Star                   | Yes            | New start/final state with $\epsilon$ -loops                  |
| Complement                    | Yes            | Swap final and non-final (complete DFA)                       |
| Intersection                  | Yes            | Product construction (or De Morgan's law)                     |
| Difference                    | Yes            | $L_1 - L_2 = L_1 \cap (\Sigma^* - L_2)$                       |
| Reversal                      | Yes            | Reverse all transitions, swap start/final                     |
| Homomorphism                  | Yes            | Replace each symbol with its image                            |
| Inverse Homomorphism          | Yes            | Standard construction                                         |
| Shuffle                       | No             | Not closed                                                    |
| Intersection with non-regular | No             | Not closed                                                    |
| Difference with non-regular   | No             | Not closed                                                    |

### 2.30 Practice Questions (Typical Exam Style)

**Q1 :** Identify which of the following are regular or not.

- $L_1 = \{w \in \{0,1\}^* \mid w \text{ ends with } 01\}$
- $L_2 = \{0^n 1^n \mid n \geq 0\}$
- $L_3 = \{w \in \{a,b\}^* \mid \text{number of } a\text{'s is even}\}$
- $L_4 = \{ww \mid w \in \{0,1\}^*\}$

**Q2 :** Prove using pumping lemma that  $L = \{0^n 1^n \mid n \geq 0\}$  is not regular.

**Steps :**

- Assume  $L$  is regular with pumping length  $p$ .
- Choose  $w = 0^p 1^p$ .
- Consider all possible decompositions  $w = xyz$ .
- Show that for some  $i$ , the pumped string  $xy^i z$  is not in  $L$ .
- Conclude  $L$  is not regular.

**Q3 :** Construct a regular expression for the language over  $\{0,1\}$ .

- All strings ending with 00.
- All strings having at least one 1.
- All strings with even number of 0's.
- All strings over  $\{0,1\}$  with no two consecutive 1's.

**Q4 :** Convert the following RE to an NFA.

- $(0 + 1)^* 01$
- $0^*(1 + 01)^*$
- $(a + b)^* abb$

**Q5 :** Convert the following grammar to an FA.

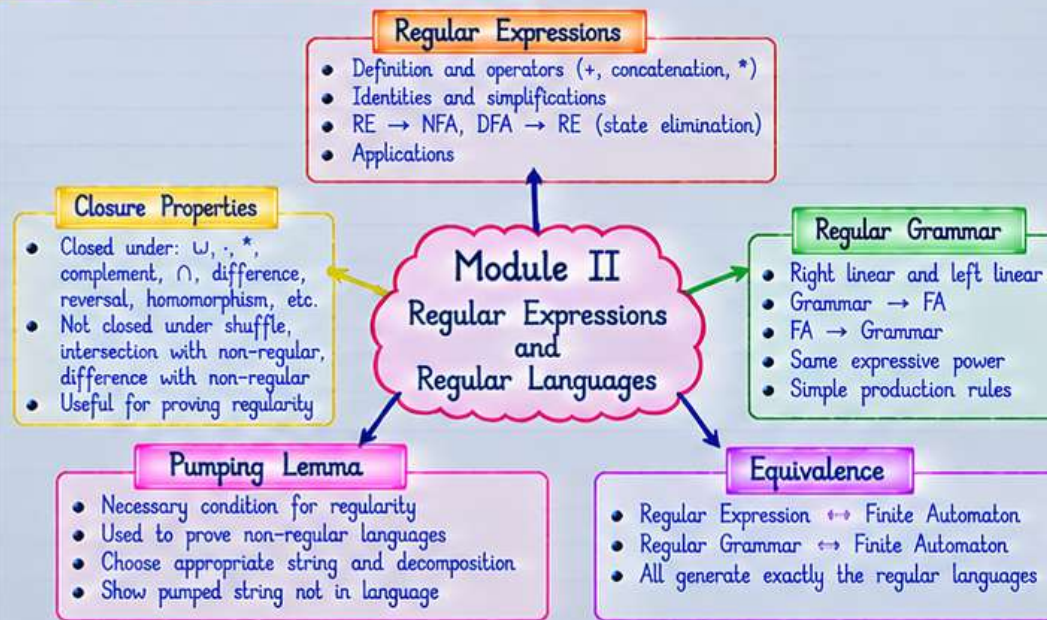
$G : S \rightarrow 0S \mid 1S \mid 01 \mid \epsilon$

Is the language regular? Justify.

**Q6 :** Show that regular languages are closed under intersection using product construction.

Given DFAs  $M_1$   $M_2$  for  $L_1$  and  $L_2$ , construct a DFA for  $L_1 \cap L_2$ .

### 2.31 Quick Revision : Mind Map





## Module - II

### Regular Grammars and Conversions

(11)

#### 2.32 Regular Grammar from a Regular Expression

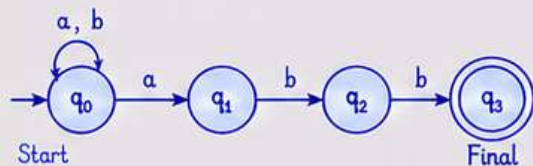
- A regular expression can be converted to a regular grammar using the syntax structure of the expression.
- We first convert the regular expression to an NFA (using Thompson's construction) and then convert the NFA to a regular grammar.
- Alternatively, we can directly construct a grammar using production rules based on operators.

**Method :**

RE  $\rightarrow$  NFA  
 $\downarrow$   
 NFA  $\rightarrow$  Regular Grammar  
 $\downarrow$   
 Simplify grammar

**Example :** Construct a regular grammar for the regular expression  $(a + b)^*abb$ .

**Step 1 : Build an NFA (sketch)**



**Note :**

- Each state is a non-terminal.
- Add a production  $A \rightarrow aB$  for every transition  $A \xrightarrow{a} B$ .
- For final state  $F$ , add  $F \rightarrow \epsilon$ .
- Simplify the grammar if needed.

**Step 2 : Convert NFA to Grammar**

| From State | Transition         | Production               |
|------------|--------------------|--------------------------|
| q0         | a $\rightarrow$ q0 | $S \rightarrow aS$       |
| q0         | b $\rightarrow$ q0 | $S \rightarrow bS$       |
| q0         | a $\rightarrow$ q1 | $S \rightarrow aA$       |
| q1         | b $\rightarrow$ q2 | $A \rightarrow bB$       |
| q2         | b $\rightarrow$ q3 | $B \rightarrow bC$       |
| q3         | —                  | $C \rightarrow \epsilon$ |

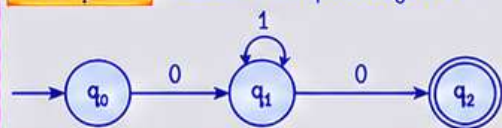
Start symbol :  $S$

Productions :  $S \rightarrow aS \mid bS \mid aA$   
 $A \rightarrow bB$   
 $B \rightarrow bC$   
 $C \rightarrow \epsilon$

#### 2.33 Regular Grammar from a DFA

- Given a DFA  $M = (Q, \Sigma, \delta, q_0, F)$ , we can construct an equivalent right linear grammar as follows :
- For each transition  $\delta(q, a) = p$ , add a production  $Q \rightarrow aP$ .
- For each final state  $f$ , add  $F \rightarrow \epsilon$ .
- The start symbol is the start state  $q_0$ .

**Example :** Consider the following DFA.



**Observation :**

- DFA  $\rightarrow$  right linear grammar (RLG).
- Every state becomes a non-terminal.
- Final states get  $\epsilon$ -production.

**Construct a regular grammar.**

| From State | Transition         | Production               |
|------------|--------------------|--------------------------|
| q0         | 0 $\rightarrow$ q1 | $S \rightarrow 0A$       |
| q1         | 1 $\rightarrow$ q1 | $A \rightarrow 1A$       |
| q1         | 0 $\rightarrow$ q2 | $A \rightarrow 0B$       |
| q2         | —                  | $B \rightarrow \epsilon$ |

Grammar :  $S \rightarrow 0A$   
 $A \rightarrow 1A \mid 0B$   
 $B \rightarrow \epsilon$

#### 2.34 Regular Grammar and Finite Automaton : Equivalence

(12)

- Regular grammars and finite automata are **equivalent** in expressive power.
- Every regular grammar generates a regular language.
- For every finite automaton (DFA/NFA), there exists a regular grammar that generates the same language.
- This establishes : Regular grammar  $\leftrightarrow$  Regular expression  $\leftrightarrow$  Finite automaton.

| Model              | Representation     | Language Type    | Equivalence To                       |
|--------------------|--------------------|------------------|--------------------------------------|
| Regular Expression | Algebraic notation | Regular language | Finite automaton, Regular grammar    |
| Finite Automaton   | (DFA / NFA)        | Regular language | Regular expression, Regular grammar  |
| Regular Grammar    | (RLG / LLG)        | Regular language | Regular expression, Finite automaton |

#### 2.35 Summary of Conversions

| From               | To                 | Method                                     | Result          |
|--------------------|--------------------|--------------------------------------------|-----------------|
| Regular Expression | NFA                | Thompson's construction                    | $\epsilon$ -NFA |
| NFA                | DFA                | Subset construction                        | Equivalent DFA  |
| DFA                | Regular Expression | State elimination                          | Equivalent RE   |
| DFA / NFA          | Regular Grammar    | Create productions from transitions        | RLG             |
| Regular Grammar    | Finite Automaton   | Create states from non-terminals           | NFA (usually)   |
| Regular Expression | Regular Grammar    | RE $\rightarrow$ NFA $\rightarrow$ Grammar | RLG             |

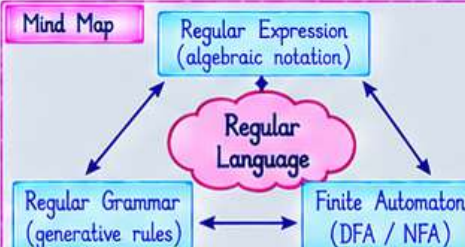
#### 2.36 Important Points to Remember

- Regular expressions, finite automata and regular grammars are equivalent.
- Regular grammar generates exactly the regular languages.
- RLG has productions of the form  $A \rightarrow aB \mid a \mid \epsilon$ .
- LLG has productions of the form  $A \rightarrow Ba \mid a \mid \epsilon$ .
- Every DFA can be converted to a right linear grammar.
- Every regular expression can be converted to a regular grammar.
- Conversion may introduce  $\epsilon$ -productions and unnecessary states; simplify the result.
- These equivalences are fundamental for compiler design and pattern processing.

**Checkpoint**

- Convert the RE  $(0+1)^*01$  to a regular grammar.
- Construct a regular grammar for strings starting with a and ending with bb.
- Given a DFA, construct an equivalent RLG.
- Explain why regular grammars and finite automata are equivalent in expressive power.

**Mind Map**





## Regular Grammar : Introduction and Types

### 2.37 Introduction to Regular Grammar

- A grammar is a set of production rules that generate a language.
- A regular grammar is a special type of grammar that generates **exactly the regular languages**.
- Regular grammars have very simple production rules.
- Every regular grammar can be converted to a finite automaton and every finite automaton can be converted to a regular grammar.

#### Key Idea :

Regular Grammar  
 $\leftrightarrow$   
 Regular Language  
 $\leftrightarrow$   
 Finite Automaton

### 2.38 Formal Definition

A grammar is a 4-tuple  $G = (V, \Sigma, P, S)$

- $V$  : finite set of non-terminals (variables)
- $\Sigma$  : finite set of terminals ( $\Sigma \cap V = \emptyset$ ).
- $P$  : finite set of production rules
- $S$  : start symbol ( $S \in V$ ).

#### Regular Grammar :

A grammar is regular if every production is of one of the following forms :

- Right Linear Grammar (RLG) :  
 $A \rightarrow aB \mid a \quad (a \in \Sigma, B \in V)$
- Left Linear Grammar (LLG) :  
 $A \rightarrow Ba \mid a \quad (a \in \Sigma, B \in V)$

### 2.39 Types of Regular Grammars

| Type                       | Production Form           | Example                                                | Language Generated |
|----------------------------|---------------------------|--------------------------------------------------------|--------------------|
| Right Linear Grammar (RLG) | $A \rightarrow aB \mid a$ | $S \rightarrow aA \mid b$<br>$A \rightarrow aA \mid b$ | Regular language   |
| Left Linear Grammar (LLG)  | $A \rightarrow Ba \mid a$ | $S \rightarrow Ab \mid a$<br>$A \rightarrow aB \mid a$ | Regular language   |

### 2.40 Example : A Right Linear Grammar

#### Grammar :

$G = (\{S, A\}, \{a, b\}, P, S)$   
 $P :$   
 $S \rightarrow aA \mid b$   
 $A \rightarrow aA \mid b$

#### Derivation Examples :

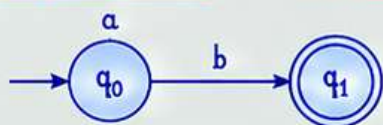
- $S \Rightarrow b$  (string :  $b$ )
  - $S \Rightarrow aA \rightarrow ab$  (string :  $ab$ )
  - $S \Rightarrow aA \rightarrow aaA \rightarrow aab$  (string :  $aab$ )
  - $S \Rightarrow aA \rightarrow aaA \rightarrow aaaA \rightarrow aaab$  (string :  $aaab$ )
- So language  $L = \{a^n b \mid n \geq 0\}$

#### Derivation Tree :

(for string  $aab$ )



#### Corresponding NFA :



#### Note :

- $S$  corresponds to  $q_0$ .
- $A$  corresponds to  $q_1$  (since it has a-loop  $a$ ).
- The rule  $S \rightarrow b$  creates transition to final state.
- Every regular grammar has an equivalent NFA.

### 2.41 Example : A Left Linear Grammar

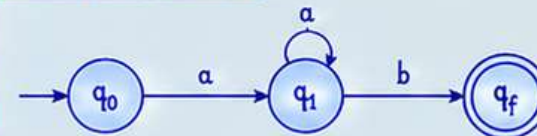
#### Grammar :

$G = (\{S, A\}, \{a, b\}, P, S)$   
 $P :$   
 $S \rightarrow Ab \mid a$   
 $A \rightarrow Ab \mid a$

#### Derivation Examples :

- $S \Rightarrow a$  (string :  $a$ )
  - $S \Rightarrow Ab \rightarrow ab$  (string :  $ab$ )
  - $S \Rightarrow Ab \rightarrow Aab \rightarrow aab$  (string :  $aab$ )
  - $S \Rightarrow Ab \rightarrow Aab \rightarrow Aaab \rightarrow aaab$
- So language  $L = \{a^n b \mid n \geq 0\}$

#### Corresponding NFA :



#### Note :

- Left linear grammar generates the same type of regular languages as right linear grammar.
- Both are equivalent in expressive power.

### 2.42 Regular Grammar for a Given Language

**Example 1 :**  $L = \{a^n b \mid n \geq 0\}$

RLG :  $S \rightarrow aS \mid b$

**Example 2 :**  $L = \{(ab)^n \mid n \geq 0\}$

RLG :  $S \rightarrow abS \mid \epsilon$

**Example 3 :**  $L = \{a, ab, abb, abbb, \dots\}$

RLG :  $S \rightarrow aA \mid a$   
 $A \rightarrow bA \mid b$

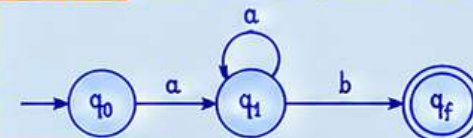
**Example 4 :**  $L = \{a^n \mid n \geq 1\}$

RLG :  $S \rightarrow aS \mid a$

### 2.43 Converting a Finite Automaton to a Regular Grammar

- Step 1 :** For each state  $q_i$ , create a non-terminal  $A_i$ .
- Step 2 :** For each transition  $\delta(q_i, a) = q_j$ , add production  $A_i \rightarrow aA_j$ .
- Step 3 :** For each final state  $q_f$ , add production  $A_f \rightarrow \epsilon$ .
- Step 4 :** The start symbol is the non-terminal corresponding to the start state.

**Example :** Construct a regular grammar for the DFA.



#### Grammar :

$V = \{A_0, A_1, A_2\}$ ,  $\Sigma = \{a, b\}$ ,  $S = A_0$   
 $P :$   
 $A_0 \rightarrow aA_1$   
 $A_1 \rightarrow aA_1 \mid bA_2$   
 $A_2 \rightarrow \epsilon$

#### General Observation :

- Every DFA has a unique (right linear) regular grammar.
- If a state is final, add  $\epsilon$ -production.
- If there are multiple transitions with same symbol, add multiple productions.

#### Note :

- The grammar obtained is always right linear.
- We can similarly construct a left linear grammar by reversing the transitions.



## Regular Expression $\rightarrow$ NFA (Thompson's Construction)

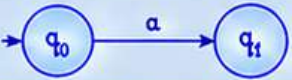
### 2.44 Overview

- Thompson's construction is a standard method to convert a regular expression into an equivalent  $\epsilon$ -NFA.
- It constructs an NFA for basic RE components ( $\emptyset$ ,  $\epsilon$ ,  $a$ ) and then combines them using rules for union ( $|$ ), concatenation and Kleene star ( $*$ ).
- The resulting NFA recognizes the same language as the given regular expression.

#### Key Idea :

- Build small NFA for basic RE components.
- Combine using standard rules.
- Always uses  $\epsilon$ -transitions.

### 2.45 Basic Constructions

| Regular Expression        | NFA Construction                                                                                                                 |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| $\emptyset$ (empty set)   | Two states with no transition.<br>             |
| $\epsilon$ (empty string) | Two states with an $\epsilon$ -transition.<br> |
| $a$ (single symbol)       | Two states with a transition on $a$ .<br>      |

### 2.46 Operations

#### 1. Union ( $R_1 | R_2$ )

- Create a new start state with  $\epsilon$ -transitions to start states of  $R_1$  and  $R_2$ .
- Create a new final state with  $\epsilon$ -transitions from final states of  $R_1$  and  $R_2$ .



#### 2. Concatenation ( $R_1 R_2$ )

- Connect final state of NFA for  $R_1$  to start state of NFA for  $R_2$  using an  $\epsilon$ -transition.



All other states remain same.

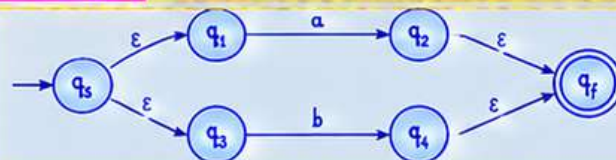
#### 3. Kleene Star ( $R^*$ )

- Create a new start and final state.
- Add  $\epsilon$ -transitions to allow zero or more occurrences.



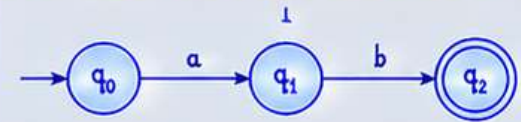
### 2.47 Worked Example 1 : RE = $a | b$

- Construct NFA for  $a$  and  $b$ .
- Apply union rule.
- Add new start and final states.



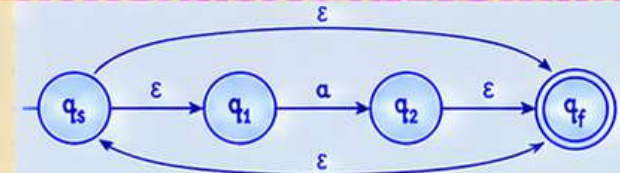
### 2.48 Worked Example 2 : RE = $ab$

- Construct NFA for  $a$  and  $b$ .
- Apply concatenation rule.
- Final NFA:



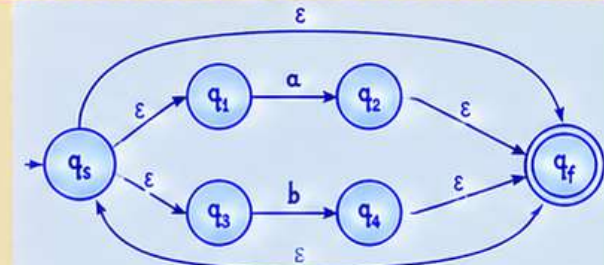
### 2.49 Worked Example 3 : RE = $a^*$

- Construct NFA for  $a$ .
- Apply Kleene star rule.
- Final NFA:



### 2.50 Worked Example 4 : RE = $(a + b)^*$

- Construct NFA for  $(a | b)$ .
- Apply Kleene star rule.
- Final NFA:



### 2.51 Important Observations

- The NFA constructed by Thompson's method has only  $\epsilon$ -transitions and symbol transitions.
- The number of states in the NFA is linear in the size of the regular expression.
- For a RE with  $n$  symbols/operators, the constructed NFA will have at most  $2n$  states.
- The construction preserves the language exactly (no extra strings).
- The resulting NFA can be converted to an equivalent DFA if needed (subset construction).

### 2.52 Practice Questions

- Q1 : Construct an  $\epsilon$ -NFA for the regular expression :
- $a | b$
  - $ab$
  - $a^*$
  - $(a | b)^*$
- Q2 : Draw the  $\epsilon$ -NFA for the regular expression :
- $a^*b$
  - $(a | b)ab$
  - $(ab)^*$
  - $a + b^*$

### 2.53 Summary

- Thompson's construction provides a systematic way to convert a regular expression to an  $\epsilon$ -NFA.
- The basic components are  $\emptyset$ ,  $\epsilon$ ,  $a$ .
- Three main operations : union, concatenation and Kleene star.
- The constructed NFA accepts exactly the same language as the given regular expression.
- This construction is widely used in compiler design and pattern matching.



## Module - II

### Finite Automata and Regular Grammar

(17)

#### 2.54 Finite Automata (FA) : Introduction

- A finite automaton is an abstract machine that accepts regular languages.
- It has a finite number of states and processes input strings symbol by symbol.
- Two main types :

- 1 DFA (Deterministic Finite Automaton)
- 2 NFA (Non-deterministic Finite Automaton)

#### Key Components :

- $Q$  : finite set of states
- $\Sigma$  : input alphabet
- $\delta$  : transition function
- $q_0$  : start state ( $q_0 \in Q$ )
- $F$  : set of final states ( $F \subseteq Q$ )
- Usually written as a 5-tuple  $(Q, \Sigma, \delta, q_0, F)$ .

#### 2.55 Deterministic Finite Automaton (DFA)

##### Definition :

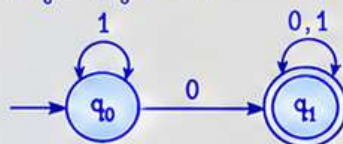
A DFA is a 5-tuple  $M = (Q, \Sigma, \delta, q_0, F)$  where  $\delta : Q \times \Sigma \rightarrow Q$  (exactly one transition for each state and input symbol).

##### Transition Table :

| State | 0     | 1     |
|-------|-------|-------|
| $q_0$ | $q_1$ | $q_0$ |
| $q_1$ | $q_1$ | $q_1$ |

##### Transition Diagram Example :

(Strings ending with 0)



Example : DFA for strings ending with 0

- $Q = \{q_0, q_1\}$
- $\Sigma = \{0, 1\}$
- $q_0$  = start state
- $F = \{q_1\}$
- Transition table as shown.

#### 2.56 Non-deterministic Finite Automaton (NFA)

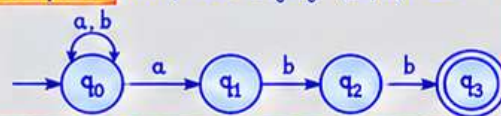
##### Definition :

An NFA is a 5-tuple  $M = (Q, \Sigma, \delta, q_0, F)$  where  $\delta : Q \times \Sigma \rightarrow 2^Q$  (a set of possible next states).

##### Transition Table :

| State | a              | b           |
|-------|----------------|-------------|
| $q_0$ | $\{q_0, q_1\}$ | $\{q_0\}$   |
| $q_1$ | $\emptyset$    | $\{q_2\}$   |
| $q_2$ | $\emptyset$    | $\{q_3\}$   |
| $q_3$ | $\emptyset$    | $\emptyset$ |

Example : NFA for the language  $(a | b)^* abb$



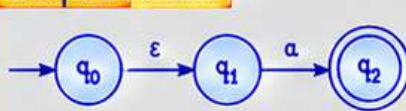
##### Important Notes :

- NFA can have multiple transitions for the same symbol.
- NFA can also have  $\epsilon$ -transitions.
- Every DFA is an NFA, but not every NFA is a DFA.
- NFA and DFA are equivalent in expressive power (both accept exactly regular languages).

#### 2.57 $\epsilon$ -NFA (NFA with $\epsilon$ -transitions)

- An  $\epsilon$ -NFA allows transitions on  $\epsilon$  (empty string).
- It does not consume any input symbol.
- Useful in simplifying constructions and in conversion from regular expressions.

Example :  $\epsilon$ -NFA



#### 2.58 Comparison : DFA vs NFA vs $\epsilon$ -NFA

(18)

| Feature                 | DFA                                      | NFA                                        | $\epsilon$ -NFA                                                |
|-------------------------|------------------------------------------|--------------------------------------------|----------------------------------------------------------------|
| Transition function     | $\delta : Q \times \Sigma \rightarrow Q$ | $\delta : Q \times \Sigma \rightarrow 2^Q$ | $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$ |
| Transitions per symbol  | Exactly one                              | Zero, one or many                          | Zero, one or many (including $\epsilon$ )                      |
| $\epsilon$ -transitions | Not allowed                              | Not allowed                                | Allowed                                                        |
| Computational power     | Regular languages                        | Regular languages                          | Regular languages                                              |
| Implementation          | Simple                                   | Easier for design                          | Useful for construction                                        |

#### 2.59 Converting NFA to DFA (Subset Construction)

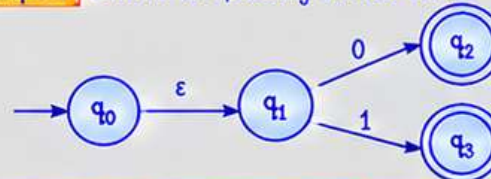
##### Steps :

- 1 Find  $\epsilon$ -closure of start state (if  $\epsilon$ -NFA).
- 2 Each DFA state is a set of NFA states.
- 3 For each input symbol, compute move and  $\epsilon$ -closure.
- 4 Continue until no new states are found.
- 5 Final states are the sets containing at least one final state of the NFA.

##### Definition :

- $\epsilon$ -closure( $q$ ) : set of states reachable from  $q$  using only  $\epsilon$ -transitions (including  $q$  itself).
- move( $S, a$ ) : set of states reachable from any state in  $S$  on input  $a$ .

Example : Convert the following NFA to DFA.



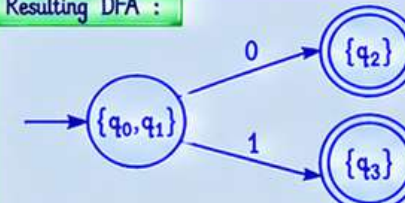
##### Step 1 : $\epsilon$ -closures

- $\epsilon$ -closure( $q_0$ ) =  $\{q_0, q_1\}$
- $\epsilon$ -closure( $q_1$ ) =  $\{q_1\}$
- $\epsilon$ -closure( $q_2$ ) =  $\{q_2\}$
- $\epsilon$ -closure( $q_3$ ) =  $\{q_3\}$

##### Step 2 : DFA States and Transitions

| DFA State (Set) | 0           | 1           |
|-----------------|-------------|-------------|
| $\{q_0, q_1\}$  | $\{q_2\}$   | $\{q_3\}$   |
| $\{q_2\}$       | $\emptyset$ | $\emptyset$ |
| $\{q_3\}$       | $\emptyset$ | $\emptyset$ |
| $\emptyset$     | $\emptyset$ | $\emptyset$ |

##### Resulting DFA :



#### 2.60 Important Observations

- 1 NFA can be converted to an equivalent DFA using subset construction.
- 2 The number of states in the resulting DFA can be at most  $2^n$  ( $n$  = number of states in NFA).
- 3  $\epsilon$ -transitions can be eliminated before applying subset construction.
- 4 DFA, NFA and  $\epsilon$ -NFA all accept exactly the regular languages.
- 5 NFAs are often easier to design, but DFAs are easier to implement.

##### Summary

- Finite automata are abstract machines for recognizing regular languages.
- DFA : one transition per symbol.
- NFA : multiple transitions allowed.
- $\epsilon$ -NFA : allows  $\epsilon$ -transitions.
- All are equivalent in power.
- NFA  $\rightarrow$  DFA using subset construction.



## Regular Grammar and Finite Automata : Conversions

### 2.68 Converting a Regular Grammar to an NFA

- A regular grammar (right or left linear) can be converted to an NFA.
- Each non-terminal becomes a state.
- For each production, add transitions according to the form:
  - $A \rightarrow aB$  : add transition  $A \rightarrow a B$
  - $A \rightarrow a$  : add transition  $A \rightarrow a$  to a new final state
  - $A \rightarrow \epsilon$  : mark  $A$  as a final state.
- The start symbol of the grammar is the start state of the NFA.

#### Algorithm :

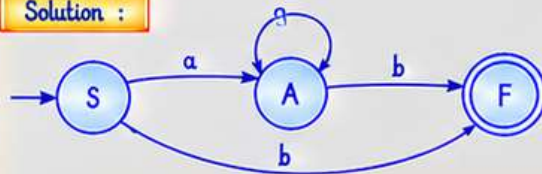
- Create a state for each non-terminal.
- For each production, add transitions.
- If  $A \rightarrow a$ , add a transition to a new final state.
- If  $A \rightarrow \epsilon$ , mark  $A$  as a final state.
- Start symbol is the start state.

**Example 1 :** Convert the grammar to NFA.

$$G : S \rightarrow aA \mid b$$

$$A \rightarrow aS \mid b$$

**Solution :**



#### Notes :

- The resulting NFA has at most  $n + 1$  states ( $n$  = number of non-terminals).
- Multiple final states may exist.
- The obtained NFA accepts exactly the same language as the grammar.
- Works for both right linear and left linear grammars (with appropriate handling).

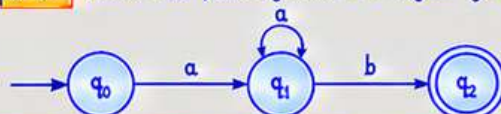
### 2.69 Converting an NFA to a Regular Grammar

- Every NFA can be converted to an equivalent regular grammar.
- Each state is a non-terminal.
- For each transition add a production.
- For each final state, add  $\epsilon$ -production.
- The start state is the start symbol of the grammar.

#### Algorithm :

- Create a non-terminal for each state.
- For each transition  $q \rightarrow p$ , add production  $Q \rightarrow aP$ .
- For each final state  $q_f$ , add  $Q_f \rightarrow \epsilon$ .
- Start symbol is the start state's non-terminal.

**Example 2 :** Convert the following NFA to a regular grammar.



**Solution :**

$$G : Q_0 \rightarrow aQ_1$$

$$Q_1 \rightarrow aQ_1 \mid bQ_2$$

$$Q_2 \rightarrow \epsilon$$

### 2.70 Left Linear Grammar to NFA

- A left linear grammar has productions of the form :  $A \rightarrow Ba$  or  $A \rightarrow a$  or  $A \rightarrow \epsilon$
- It can be converted to an NFA by reversing the direction of transitions compared to RLG conversion.

**Example :**  $G : S \rightarrow Aa \mid b$

$$A \rightarrow Sb \mid \epsilon$$

### 2.71 Worked Example : Grammar to NFA

Convert the following regular grammar to NFA.

$$G : S \rightarrow aA \mid b$$

$$A \rightarrow bA \mid a$$

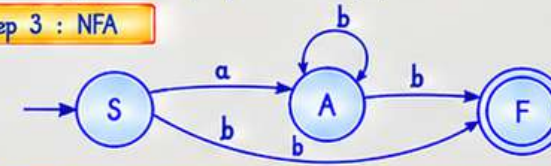
**Solution :**

**Step 1 : States**

| Non-terminal | State     |
|--------------|-----------|
| S            | S (start) |
| A            | A         |

Add a new final state F for productions of form  $A \rightarrow a$ .

**Step 3 : NFA**



**Step 2 : Transitions**

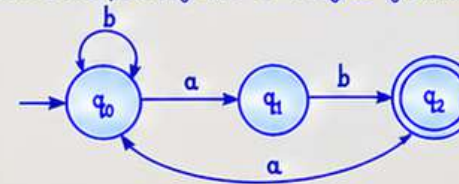
| Production         | Transition          |
|--------------------|---------------------|
| $S \rightarrow aA$ | $S \rightarrow a A$ |
| $S \rightarrow b$  | $S \rightarrow b F$ |
| $A \rightarrow bA$ | $A \rightarrow b A$ |
| $A \rightarrow a$  | $A \rightarrow a F$ |

**Result :**

This NFA accepts the same language as the given grammar.

### 2.72 Worked Example : NFA to Grammar

Convert the following NFA to a regular grammar.



**Solution :** Let  $Q_0, Q_1, Q_2$  be non-terminals.

Productions :  $Q_0 \rightarrow bQ_0 \mid aQ_1$

$$Q_1 \rightarrow bQ_2$$

$$Q_2 \rightarrow aQ_0 \mid \epsilon$$

Start symbol :  $Q_0$

This grammar generates the same language as the NFA.

### 2.73 Summary of Conversions

| From                | To                  | Main Idea                          | Key Rules                                                                                                     |
|---------------------|---------------------|------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Regular Grammar     | NFA                 | Non-terminals $\rightarrow$ states | $A \rightarrow aB \Rightarrow A \rightarrow a B_i B$<br>$A \rightarrow a \Rightarrow A \rightarrow a F$       |
| NFA                 | Regular Grammar     | States $\rightarrow$ non-terminals | $q \rightarrow p \Rightarrow Q \rightarrow aP; q_f \text{ final} \Rightarrow Q_f \rightarrow \epsilon$        |
| Left Linear Grammar | NFA                 | Reverse transitions                | $A \rightarrow Ba \Rightarrow A \rightarrow a B$ (reverse)<br>$A \rightarrow a \Rightarrow A \rightarrow a F$ |
| NFA                 | Left Linear Grammar | Reverse and convert                | $q \rightarrow p \Rightarrow P \rightarrow aQ; q_f \text{ final} \Rightarrow Q_f \rightarrow \epsilon$        |

### 2.74 Important Points to Remember

- Regular grammar and finite automata are equivalent models.
- Every regular grammar has an equivalent NFA, and vice-versa.
- Right linear grammars are easier to convert to NFA.
- In NFA  $\rightarrow$  Grammar conversion, one production is added per per transition.
- $\epsilon$ -productions are added for final states.
- The obtained grammar generates exactly the same language as the given NFA.
- These conversions are fundamental for compiler design and formal language theory.



## Module - II

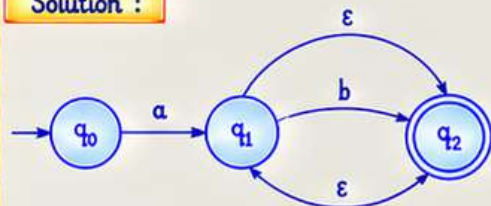
### Integrated Problem Solving (Exam Practice)

(23)

**Q1** Construct an NFA for the following regular expression.

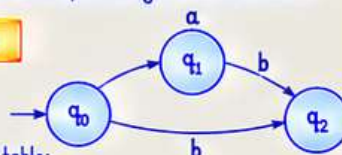
(a)  $a(b | \epsilon)^*$

**Solution :**



**Q2** Convert the following NFA to DFA.

**Solution :**



DFA transition table:

| State             | a                 | b                 |
|-------------------|-------------------|-------------------|
| {q <sub>0</sub> } | {q <sub>1</sub> } | {q <sub>2</sub> } |
| {q <sub>1</sub> } | {q <sub>1</sub> } | {q <sub>2</sub> } |
| {q <sub>2</sub> } | ∅                 | ∅                 |
| ∅                 | ∅                 | ∅                 |

**Q3** Convert the following regular expression to a regular grammar.

$R = (a | b)^* abb$

**Solution :**

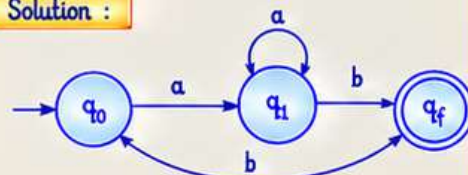
Let  $G = (S, A, B, C, D, \Sigma, P)$

Productions :  $S \rightarrow aS | bS | aA$   
 $A \rightarrow bB$   
 $B \rightarrow bC$   
 $C \rightarrow \epsilon$

**Q4** Convert the following grammar to an NFA.

$G : S \rightarrow aA | b$   
 $A \rightarrow aA | b$

**Solution :**



**Q5** Use pumping lemma to show that the language  $L = \{a^n b^n | n \geq 0\}$  is not regular.

**Solution :**

- Assume  $L$  is regular with pumping length  $p$ .
- Take  $w = a^p b^p \in L$ .
- Let  $w = xyz$  with  $|xy| \leq p$ ,  $|y| \geq 1$ .
- Then  $y = a^k$  ( $k \geq 1$ ).
- Pump  $i = 0$  :  $xz = a^{p-k} b^p \notin L$ .
- Contradiction. Hence  $L$  is not regular.

**Q6** Check whether the following language is regular or not.  
 $L = \{w \in \{0,1\}^* | \text{number of 0's} = \text{number of 1's}\}$

**Solution :**

- Use pumping lemma :
- Take  $w = 0^p 1^p$ .
- $|xy| \leq p \Rightarrow y = 0^k$  ( $k \geq 1$ ).
- Pump  $i = 0 \Rightarrow 0^{p-k} 1^p$  (not in  $L$ ).
- Hence language is not regular.

#### Additional Practice Questions

- Convert  $(a + b)^* a(a + b)$  to NFA using Thompson's construction.
- Convert the given NFA to an equivalent DFA and minimize it.
- Convert the DFA to a regular expression using state elimination.
- Construct a right linear grammar for the language  $L = \{w | w \text{ ends with } 01\}$ .
- Prove that the language  $\{ww | w \in \{0,1\}^*\}$  is not regular using pumping lemma.
- Show that the class of regular languages is closed under complement.

## Module - II

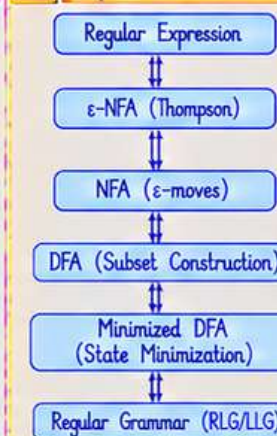
### Final Revision & Formula Sheet

(24)

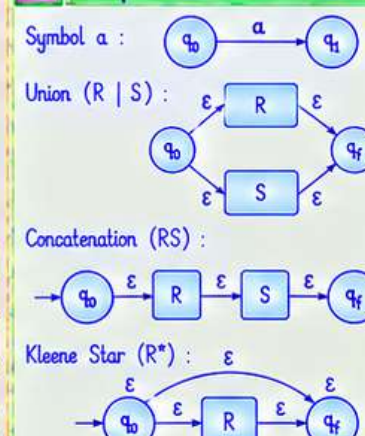
#### 1 Regular Expression Identities

- $R + \emptyset = R$
- $R + \epsilon = R$  (if  $\epsilon \in R$ )
- $R + R = R$
- $R\emptyset = \emptyset R = \emptyset$
- $R\epsilon = \epsilon R = R$
- $(R)^* = \epsilon + RR^* = \epsilon + R^*R$
- $(R^*)^* = R^*$
- $\emptyset^* = \epsilon$
- $(R + S)^* = (R^* S^*)^*$
- $(RS)^* = \epsilon + RS(RS)^*$

#### 2 Important Conversions



#### 3 Thompson's Construction Rules



#### 4 Pumping Lemma (Regular Languages)

If  $L$  is regular,  $\exists p \geq 1$  such that every  $w \in L$  with  $|w| \geq p$  can be written as  $w = xyz$  where

- $|xy| \leq p$
- $|y| \geq 1$
- $\forall i \geq 0, xy^i z \in L$

#### 5 Closure Properties (Regular Languages)

- Closed under : union, concatenation, Kleene star, complement, intersection, difference, reversal, homomorphism, inverse homomorphism, left/right quotient.
- Not closed under : (proven using pumping lemma)
  - Intersection with non-regular language
  - Some operations with context-free languages.

#### 6 DFA Minimization Steps

- Remove unreachable states.
- Mark final and non-final states.
- Iteratively mark distinguishable pairs.
- Merge unmarked pairs.
- Obtain minimized DFA.

#### 7 Grammar Conversion Rules

| Regular Grammar $\rightarrow$ NFA               | NFA $\rightarrow$ Regular Grammar                                     |
|-------------------------------------------------|-----------------------------------------------------------------------|
| $S \rightarrow aA : q_s \xrightarrow{a} q_A$    | For each transition $\delta(q_i, a) = q_j$ add $q_i \rightarrow aq_j$ |
| $S \rightarrow a : q_s \xrightarrow{a} q_f$     | For each final state $q_f$ add $q_f \rightarrow \epsilon$             |
| $S \rightarrow \epsilon : q_s \text{ is final}$ | Start symbol is start state.                                          |

#### 8 Types of Regular Grammar

| Type                 | Form                                             | Example                    |
|----------------------|--------------------------------------------------|----------------------------|
| Right Linear Grammar | $A \rightarrow aB   a$                           | Generates regular language |
| Left Linear Grammar  | $A \rightarrow Ba   a$<br>$S \rightarrow aS   b$ | Generates regular language |

#### 9 Key Differences : DFA vs NFA

| Feature                 | DFA         | NFA                        |
|-------------------------|-------------|----------------------------|
| Transition              | Exactly one | Zero, one or many          |
| $\epsilon$ -transitions | Not allowed | Allowed ( $\epsilon$ -NFA) |
| Computational power     | Same        | Same (Regular)             |
| Implementation          | Easier      | Often easier to design     |

#### 10 Common Mistakes

- Forgetting  $\epsilon$  in Thompson's construction.
- Incorrect subset construction (missing states).
- Not removing unreachable states.
- Wrong marking in DFA minimization.
- Confusing RLG and LLG rules.
- Incorrect application of pumping lemma.
- Missing  $\epsilon$ -productions for final states.

#### 11 Exam Tips

- Draw clear state diagrams with proper  $\epsilon$  labels.
- Always write transition tables.
- Show all steps in conversions.
- For pumping lemma, choose a correct  $w$  and justify  $|xy| \leq p$ .
- Use closure properties to quickly prove non-regularity.
- Practice mixed problems (RE, FA, Grammar).
- Write final answer clearly with a proper diagram / grammar.